STG 时空图谱记忆系统 —— 核心逻辑详解

目录

1. 系统总览
2. 向量化逻辑
3. 更新逻辑
4. 查询逻辑
5. LLM 接入指南
6. 当前无 LLM 的问答原理

---

1. 系统总览

1.1 架构图

```
场景图 JSON ──► [逐帧处理循环]
                    │
                    ├─► ImmediateUpdater (即时更新)
                    │       ├─ EntityTracker: 匈牙利匹配 → 实体身份关联
                    │       ├─ EventGenerator: 规则模板 → 事件描述(自然语言)
                    │       ├─ EmbeddingManager: 文本 → 384维向量
                    │       └─ FAISSStore: 向量 + 元数据 写入索引
                    │
                    └─► BufferUpdater (缓存更新, 每N帧flush)
                            ├─ MotionAnalyzer: 轨迹几何分析 → 运动模式
                            ├─ EventGenerator: 轨迹/交互 → 事件描述
                            └─ FAISSStore: 向量 + 元数据 写入索引

查询时:
  用户问题(文本) ──► EmbeddingManager → 384维向量
                 ──► FAISS.search(内积) → top-k 最相似的事件/实体元数据
                 ──► 返回结构化结果 / 拼接为 LLM context
```

1.2 数据流

```
输入: scene_graphs_target.json
      │
      ▼
  帧0 ─► 帧1 ─► 帧2 ─► ... ─► 帧N
  │       │       │               │
  ▼       ▼       ▼               ▼
[即时更新] 每帧产出:              [缓存flush] 每N帧产出:
  - entity_appeared 事件            - trajectory_summary 事件
  - entity_moved 事件               - interaction 事件
  - relation_changed 事件
  - attribute_changed 事件
```

```
    - entity_disappeared 事件
        │                              │
        ▼                              ▼
    ┌──────────────────────────────────────┐
    │          FAISS 向量索引              │
    │  ┌──────────────┐  ┌──────────────┐  │
    │  │ events 索引  │  │ entity_* 索引 │  │
    │  │ (所有事件)   │  │ (每实体一个)  │  │
    │  └──────────────┘  └──────────────┘  │
    └──────────────────────────────────────┘
```

---

2. 向量化逻辑

2.1 核心思想

系统将 **结构化的场景图数据** 转化为 **自然语言文本描述**，再用 sentence-transformers 模型将文本映射到 **384维向量空间**。这样就可以用 **语义相似度** 来检索信息，而非关键词匹配。

2.2 Embedding 模型

配置项	默认值	说明
模型名	`all-MiniLM-L6-v2`	sentence-transformers 轻量模型
向量维度	384	固定维度
归一化	是	输出 L2 归一化，使内积 = 余弦相似度
加载方式	懒加载	首次调用 `embed()` 时才加载模型

代码位置: `stg/utils.py` → `EmbeddingManager` 类

```python
class EmbeddingManager:
    def embed(self, text: str) -> np.ndarray:
        # 1. 懒加载 SentenceTransformer 模型
        # 2. model.encode(text, normalize_embeddings=True)
        # 3. 返回归一化后的 384维 float32 向量

    def embed_batch(self, texts: List[str]) -> np.ndarray:
        # 批量编码，效率更高

    def cosine_similarity(self, emb_a, emb_b) -> float:
        # 因为已归一化，直接点积 = 余弦相似度
```

2.3 什么文本被向量化?

系统中有 **两类内容** 会被转换为向量并写入 FAISS:

(A) 事件描述 → 写入 `events` 索引

每个事件由 `EventGenerator` 生成**自然语言 summary**，然后向量化：

事件类型	summary 模板示例
`entity_appeared`	`"player_1 (a person) appeared at frame 0, described as: wearing white jersey"`
`entity_moved`	`"basketball_1 moved from (320,180) to (450,220), displacement 142px at frame 10"`
`relation_changed`	`"player_1 relation changes at frame 20: new relation: holding; lost relation: near"`
`attribute_changed`	`"player_1 appearance changed at frame 30: from 'standing' to 'jumping'"`
`entity_disappeared`	`"player_1 (a person) disappeared from the scene at frame 50"`
`trajectory_summary`	`"player_1: moved to the right during frame 0 to 40, displacement 300px"`
`interaction`	`"player_1 and basketball_1 moved closer to each other (distance: 200px -> 50px)"`

关键: 这些 summary 是由**规则模板**生成的，不是 LLM 生成的。模板代码在 `stg/event_generator.py`。

(B) 实体状态描述 → 写入 `entity_{id}` 索引

每次实体状态更新时，`entity_state_description()` 函数为其生成描述文本：

```
"player_1 (a person); appears as: wearing white jersey, jumping;
 moved from (100,200) to (400,300), total displacement: 350px over frames 0-40"
```

代码位置: `stg/utils.py` → `entity_state_description()`

2.4 FAISS 索引结构

```
faiss/ 目录
├── {sample_id}_events.index          # 所有事件的向量（IndexFlatIP）
├── {sample_id}_events_meta.json      # 事件元数据（与向量一一对应）
├── {sample_id}_entity_0001.index     # 实体1的状态向量
├── {sample_id}_entity_0001_meta.json # 实体1的状态元数据
├── {sample_id}_entity_0002.index     # 实体2的状态向量
└── ...
```

- **IndexFlatIP**: FAISS 的内积索引（Inner Product），对归一化向量等价于余弦相似度
- 每个 `add_memory()` 调用都会：
  1. 将 embedding 归一化
  2. 添加到 FAISS 索引（`index.add(emb)`）

3. 将元数据 append 到对应的 metadata 列表
4. 向量在索引中的顺序与元数据列表顺序严格一致

2.5 向量化流程图

```
场景图对象                    EventGenerator                EmbeddingManager
FAISSStore
    │                            │                             │
│
    │  实体匹配/对比后            │                             │
│
    ├──────────────────────────► │                             │
│
    │                            │ 规则模板生成                  │
│
    │                            │ summary 文本                  │
│
    │                            ├────────────────────────────►│
│
    │                            │                             │ encode(text)
│
    │                            │                             │ → 384维向量
│
    │                            │
├──────────────────────►│
    │                            │                             │
│ add(向量, 元数据)
    │                            │                             │
│ → 写入索引
```

---

3. 更新逻辑

更新分为 **即时更新** 和 **缓存更新** 两层。

3.1 即时更新 (ImmediateUpdater)

触发条件: 每处理一帧就执行。

代码位置: `stg/immediate_update.py` → `ImmediateUpdater.process_frame()`

3.1.1 首帧处理

```
输入: frame_data = {"image_path": "...", "objects": [...]}
      │
      ▼
  过滤低置信度对象 (score < 0.35 的丢弃)
      │
      ▼
  所有对象注册为新实体 (EntityTracker.process_frame, prev_objects=None)
```

```
        │
        ├─► 为每个对象分配 entity_id: entity_0001, entity_0002, ...
        ├─► 创建 EntityRecord: 记录 label, tag, box, attributes, relations
        │
        ▼
    生成事件:
        ├─► gen_initial_scene_description: "Initial scene contains 65 objects: ..."
        └─► gen_entity_appeared × N: "player_1 (a person) appeared at frame 0"
        │
        ▼
    每个事件: summary → embed() → add_memory("events") → 写入 FAISS
    每个实体: state_description → embed() → add_memory("entity_{id}") → 写入 FAISS
```

3.1.2 非首帧处理

```
  输入: 当前帧 objects + 上一帧 objects
        │
        ▼
    ┌──────────────────────────────────────┐
    │ EntityTracker.match_entities()        │
    │                                       │
    │  1. IoU 矩阵 [上帧M个 × 当帧N个]      │
    │     iou_matrix[i,j] = IoU(box_i, box_j)│
    │                                       │
    │  2. Label 相似度矩阵 [M × N]          │
    │     sim_matrix[i,j] = cosine(          │
    │       embed("person"), embed("player")) │
    │     (带缓存, 同 label 不重复计算)      │
    │                                       │
    │  3. 综合得分矩阵                       │
    │     cost = α × IoU + (1-α) × label_sim │
    │     默认 α = 0.5                       │
    │                                       │
    │  4. 匈牙利算法求最优匹配               │
    │     (prev_idx, curr_idx, score)        │
    │     过滤: score ≥ 0.4 且 label_sim ≥ 0.35│
    │                                       │
    │  输出:                                 │
    │   matched:     [(prev_idx, curr_idx)]  │
    │   unmatched_prev: [消失的索引]         │
    │   unmatched_curr: [新出现的索引]       │
    └──────────────────────────────────────┘
        │
        ▼
    对每个 matched 实体:
        │
        ├─► 计算位移 = distance(prev_center, curr_center)
        │   如果 > 10px → gen_entity_moved 事件 → embed → FAISS
        │
        ├─► 对比关系: diff_relations(prev_obj, curr_obj)
        │   如果有 added/removed 关系 → gen_relation_changed 事件 → embed → FAISS
```

```
        │
        ├─► 对比属性: prev_attrs vs curr_attrs
        │   如果属性文本变化且 cosine_sim(embed(prev), embed(curr)) < 0.85
        │   → gen_attribute_changed 事件 → embed → FAISS
        │
        └─► 更新 EntityRecord: 轨迹追加、属性历史追加、关系历史追加
             更新实体状态向量 → entity_{id} 索引
        │
    对每个 unmatched_curr (新出现):
        └─► 注册新实体 → gen_entity_appeared → embed → FAISS
        │
    对每个 unmatched_prev (消失):
        └─► 首次消失时 → gen_entity_disappeared → embed → FAISS
```

3.1.3 实体匹配的数学细节

给定上一帧 M 个对象、当前帧 N 个对象:

$$ \text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} $$

$$ \text{label_sim}(i, j) = \cos(\mathbf{e}_{\text{label}*i}, \mathbf{e}*{\text{label}_j}) = \frac{\mathbf{e}_i \cdot \mathbf{e}_j}{|\mathbf{e}_i| |\mathbf{e}_j|} $$

$$ \text{score}(i, j) = \alpha \cdot \text{IoU}(i,j) + (1 - \alpha) \cdot \text{label_sim}(i,j) $$

匈牙利算法求解： $$ \min_{\pi} \sum_{i} -\text{score}(i, \pi(i)) $$ 其中 π 是二部图的最优匹配排列。

过滤条件:

- $\text{score}(i, \pi(i)) \geq 0.4$（综合阈值）
- $\text{label_sim}(i, \pi(i)) \geq 0.35$（label兜底）

3.2 缓存更新 (BufferUpdater)

触发条件: 每积累 `buffer_size`（默认5）帧的观测数据后自动 flush，以及处理完所有帧后最终 flush。

代码位置: `stg/buffer_update.py` → `BufferUpdater`

```
每帧的 buffered_observations 进入缓冲区
        │
        ▼
  缓冲区满 (≥ buffer_size)?
        │
        ├─ 否 → 继续累积
        │
        └─ 是 → flush()
                 │
                 ▼
            Step 1: 收集实体轨迹片段
                 从缓冲区 N 帧中提取每个 entity_id 在这段时间的 center + box
                 │
                 ▼
```

```
            Step 2: 轨迹分析 (MotionAnalyzer.analyze_single_entity)
                对每个动态实体:
                ├─ 计算总位移、平均速度、主方向
                ├─ 分类运动模式: stationary / moving_left / moving_right /
                │                 moving_up / moving_down / jumping /
direction_change
                └─ 如果位移 > movement_threshold → gen_trajectory_summary 事件
                │
                ▼
            Step 3: 交互分析 (MotionAnalyzer.analyze_all_interactions)
                遍历所有动态实体对 (entity_a, entity_b):
                ├─ 找到共同出现的帧
                ├─ 比较起始距离 vs 结束距离
                │   dist_ratio = dist_end / dist_start
                │   ratio < 0.7 → approaching_each_other
                │   ratio > 1.43 → departing_from_each_other
                │   方向差 < 30° 且距离稳定 → moving_together
                └─ gen_interaction_event 事件
                │
                ▼
            Step 4: 所有事件 → embed → 写入 FAISS events 索引
```

3.3 更新机制对比

特性	即时更新	缓存更新
触发频率	每帧	每 N 帧
关注点	帧间差异（移动、关系、属性）	跨帧趋势（轨迹、交互）
事件类型	appeared/moved/relation_changed/attribute_changed/disappeared	trajectory_summary/interaction
分析方法	直接比较（IoU、diff）	几何分析（方向、速度、模式）
实体状态	每帧更新实体向量	不额外更新实体向量

4. 查询逻辑

4.1 查询流程

代码位置: `stg/memory_manager.py` → `STGraphMemory.search()`

```
用户查询: "player_1 在做什么?"
      │
      ▼
  EmbeddingManager.embed(query)
      → 384维 query 向量 q
      │
      ▼
  ┌──────────────────────────────────────┐
  │ Step 1: 搜索 events 索引             │
  │   FAISS.search(q, top_k=10)          │
  │   IndexFlatIP 暴力内积搜索           │
  │   返回: [(score, idx), ...] 按相似度降序│
  │   过滤: score ≥ similarity_threshold │
  │   用 idx 从 metadata_store 取元数据  │
  └──────────────────────────────────────┘
      │
      ▼
  ┌──────────────────────────────────────┐
  │ Step 2: 搜索所有 entity_* 索引       │
  │   遍历所有 entity_{id} 索引          │
  │   对每个索引执行同样的 FAISS.search  │
  │   合并所有结果, 按 score 排序        │
  │   取 top_k/2 个最相关实体状态        │
  └──────────────────────────────────────┘
      │
      ▼
  合并结果:
    {
      "events": [
        {"score": 0.82, "metadata": {"event_type": "entity_moved", "summary":
"..."}},
        ...
      ],
      "entities": [
        {"score": 0.75, "metadata": {"tag": "player_1", "description": "..."}},
        ...
      ],
      "combined_text": "=== Related Events ===\n...\n=== Related Entities
===\n..."
    }
```

4.2 相似度计算

因为向量全部 L2 归一化，FAISS IndexFlatIP 的内积搜索等价于余弦相似度：

$$ \text{similarity}(\mathbf{q}, \mathbf{v}_i) = \mathbf{q} \cdot \mathbf{v}i = \cos(\theta{q,v_i}) $$

取值范围 $[-1, 1]$，越接近 1 表示语义越相似。

4.3 `get_context_for_qa()` — 为 LLM 准备的接口

代码位置: `stg/memory_manager.py` → `get_context_for_qa()`

该方法在 `search()` 基础上，将检索结果**格式化为结构化文本**，可直接拼接到 LLM prompt 中：

```
=== Spatio-Temporal Memory Context ===
Query: what happened to the basketball?

--- Relevant Events ---
1. [entity_moved] basketball_1 moved from (320,180) to (450,220), displacement
142px at frame 10 (frames: [10, 10])
2. [relation_changed] player_1 relation changes at frame 20: new relation: holding
(frames: [20, 20])
3. [interaction] player_1 and basketball_1 moved closer to each other (frames: [0,
40])

--- Relevant Entity States ---
1. basketball_1 - basketball_1 (a basketball); appears as: orange round ball;
moved from (320,180) to (450,220)

Scene contains 120 tracked entities (80 static, 40 dynamic)
```

---

5. LLM 接入指南

5.1 接入方式

STG 系统 **不直接集成 LLM**，而是提供 **检索增强（RAG）接口**。接入方式：

```python
from stg.memory_manager import STGraphMemory
from stg.config import STGConfig

# 1. 加载已构建的 STG
config = STGConfig(output_dir="./stg_output", faiss_dir="./stg_output/faiss")
stg = STGraphMemory(config)

# 2. 获取检索上下文
question = "视频中的球员做了什么动作?"
context = stg.get_context_for_qa(question, sample_id="video_001", top_k=10)

# 3. 拼接 prompt 并调用 LLM
from openai import OpenAI

client = OpenAI(base_url="your_api_url", api_key="your_key")
```

```
prompt = f"""You are a video understanding assistant. Based on the following
spatio-temporal memory context retrieved from a video analysis system, answer the
user's question.

{context}

Question: {question}

Please provide a detailed answer based on the context above."""

response = client.chat.completions.create(
    model="gpt-4",  # or any model
    messages=[
        {"role": "system", "content": "You are a helpful video analysis
assistant."},
        {"role": "user", "content": prompt}
    ]
)

print(response.choices[0].message.content)
```

5.2 完整测试脚本示例

创建 `scripts/llm_qa.py`:

```
"""
LLM + STG 联合问答示例脚本

Usage:
    python scripts/llm_qa.py \
        --sample_id video_001 \
        --output_dir stg_output \
        --api_base "https://api.openai.com/v1" \
        --api_key "sk-xxx" \
        --model "gpt-4" \
        --question "What is the player doing?"
"""
import argparse
from openai import OpenAI
from stg.config import STGConfig, EmbeddingConfig
from stg.memory_manager import STGraphMemory

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--sample_id", default="video_001")
    parser.add_argument("--output_dir", default="./stg_output")
    parser.add_argument("--api_base", required=True, help="LLM API base URL")
    parser.add_argument("--api_key", required=True)
    parser.add_argument("--model", default="gpt-4")
    parser.add_argument("--question", required=True)
    parser.add_argument("--top_k", type=int, default=10)
```

```
    args = parser.parse_args()

    # 加载 STG
    config = STGConfig(
        output_dir=args.output_dir,
        faiss_dir=f"{args.output_dir}/faiss"
    )
    stg = STGraphMemory(config)

    # 检索
    context = stg.get_context_for_qa(args.question, args.sample_id,
top_k=args.top_k)
    print("=== Retrieved Context ===")
    print(context)
    print()

    # 调用 LLM
    client = OpenAI(base_url=args.api_base, api_key=args.api_key)

    prompt = f"""Based on the following spatio-temporal memory retrieved from
video analysis, answer the question.

{context}

Question: {args.question}

Answer:"""

    response = client.chat.completions.create(
        model=args.model,
        messages=[
            {"role": "system", "content": "You are a video understanding
assistant. Answer questions based on the provided spatio-temporal context."},
            {"role": "user", "content": prompt}
        ],
        temperature=0.3
    )

    print("=== LLM Answer ===")
    print(response.choices[0].message.content)

if __name__ == "__main__":
    main()
```

5.3 使用本地模型（Ollama 等）

```
# Ollama 示例（兼容 OpenAI API）
client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama"  # Ollama 不需要真正的key
)
```

```
response = client.chat.completions.create(
    model="llama3",
    messages=[...],
    temperature=0.3
)
```

5.4 工作流程图

```
                    用户问题
                       │
                       ▼
            ┌──────────────────┐
            │ STG.search()     │ ← 语义检索
            │ 或               │
            │ get_context_for  │
            │ _qa()            │
            └──────────────────┘
                       │ 结构化上下文文本
                       ▼
            ┌──────────────────┐
            │   拼接 Prompt    │
            │ system + context│
            │ + question       │
            └──────────────────┘
                       │
                       ▼
            ┌──────────────────┐
            │   LLM 推理       │ ← GPT-4 / Llama / Qwen 等
            └──────────────────┘
                       │
                       ▼
                   最终答案
```

---

6. 当前无 LLM 的问答原理

6.1 核心解释

当前系统没有接入任何 LLM，`query_stg.py` 的"问答"本质上是向量检索（retrieval），不是生成式问答（generation）。

具体流程：

1. 用户输入查询文本（如 `"What is the player doing near the basketball hoop?"`）
2. `EmbeddingManager` 将查询文本编码为 384 维向量
3. FAISS 用**内积相似度**在预存的事件/实体向量中搜索最相似的 top-k 条
4. 返回这些条目的**元数据**（包括 summary 描述文本、涉及的实体、帧范围等）
5. 直接展示这些元数据作为"答案"

6.2 "答案"来源

```
用户问: "What is the player doing near the basketball hoop?"
        │
        ▼
  encode → [0.12, -0.04, 0.33, ...]  (384维)
        │
        ▼
  FAISS 内积搜索 events 索引中预存的向量
        │
        ▼
  找到语义最相似的 summary, 例如:
    score=0.72: "player_1 moved from (320,500) to (450,180), displacement 350px at
frame 30"
    score=0.68: "player_1 and basketball_hoop_1 moved closer to each other ..."
    score=0.61: "player_1 relation changes at frame 35: new relation: near"
```

这些 summary 文本是在 build 阶段由规则模板自动生成的，不是 LLM 生成的。它们的来源是：

summary 内容	数据来源	生成方式
实体出现/消失	实体匹配的 unmatched 结果	`EventGenerator` 模板
位置移动描述	bbox 中心点坐标差	坐标格式化字符串
关系变化	`subject_relations` / `object_relations` 的集合差	`diff_relations()` → 模板
轨迹摘要	多帧 center 坐标序列	`MotionAnalyzer` 几何分析 + 模板
交互描述	两实体间距离变化比	距离计算 + 阈值判断 + 模板

6.3 与接入 LLM 后的区别

方面	当前（无 LLM）	接入 LLM 后
输出形式	原始检索结果列表	自然语言答案
理解能力	仅语义相似度匹配	可理解复杂问题、做推理
回答质量	返回与问题最相关的事件描述	综合多条检索结果给出流畅回答
适用场景	调试验证、查看原始数据	最终用户交互
示例输出	`[0.72] [entity_moved] player_1 moved from...`	`"球员从左侧跑到篮筐附近并完成投篮"`

6.4 示意图

```
当前模式（纯检索）：
  Query → embed → FAISS search → 返回匹配的 metadata 列表 ✓

接入 LLM 后（RAG模式）：
  Query → embed → FAISS search → 上下文文本 → LLM 生成 → 自然语言答案 ✓
```

附录：关键配置参数速查

参数	位置	默认值	含义
`alpha`	EntityMatchingConfig	0.5	IoU 权重（实体匹配中）
`combined_threshold`	EntityMatchingConfig	0.4	综合匹配分阈值
`score_filter`	EntityMatchingConfig	0.35	检测置信度过滤
`movement_threshold`	TrajectoryConfig	10.0px	超过此位移才记录轨迹/触发 moved 事件
`static_threshold`	TrajectoryConfig	15.0px	低于此总位移标记为静态
`buffer_size`	BufferConfig	5 帧	缓冲区大小（触发 flush）
`similarity_threshold`	STGConfig	0.85	FAISS 检索相似度阈值
`direction_change_angle`	MotionConfig	45°	方向变化判定角度
`approach_distance_ratio`	MotionConfig	0.7	距离缩小比例判定为接近